

Contents

Service Architecture — Core Domain Service, Internal Auth, Multi-Region1

1. The question this note answers

2. Vocabulary — the patterns by name

3. Current state at eng-labs

4. Target (“ideal”) architecture

5. “Share the logic” vs. “share the client” — the nuance that matters most

6. Security angle — what “private Cloud Run” actually buys you

7. Multi-region — why the database is the hard part, not the backend services

8. Recap — what was verified vs. assumed, across the whole discussion

9. If this were ever acted on — rough order of operations

10. Cloud Run building block: Service vs. Job vs. Function

11

12

Service Architecture — Core Domain Service, Internal Auth, Multi-Region

- **Date:** 2026-09-03
- **Context:** A Claude Code session walking the real satellite-service architecture of the eng-labs platform (`apps/guide-agent`, `apps/manuscript-reviewer`, `apps/interview-service` around the core `apps/api`) and generalizing it into reusable architecture principles.
- **Companion notes:** [eng-labs-platform.md](#) — the monorepo/runtime source of truth. [multi-tenancy.md](#) — the auth/authorization concepts this note assumes (three-gate contract, enforcement layers).

The one-line answer: one service (the **Core Domain Service**) should own the database and the authorization rules; every other backend service should be a stateless caller of it, reached from the frontend only through it, and trusted via platform-level identity (Cloud Run IAM + short-lived tokens) — not a static shared secret, and not a second copy of the database logic in another language.

1. The question this note answers

Given a platform with one main backend (`apps/api`) plus several specialized backend workloads (an LLM agent, a batch PDF job, a real-time audio service), each possibly in a different language:

1. Should each service have its own direct database connection, or should one service own the data and expose it via an API?
2. If multiple languages need “the same” auth/DB logic, do you rewrite that logic per language, or share something else?
3. Does the frontend ever call a specialized backend service directly, or always through the core API?
4. How do you secure calls *between* backend services without assuming they share a network (VPC)?
5. What changes if this has to run in more than one geographic region?

The short answers, in order: **one owner + API, share a generated client, not the logic, always through the core API except a narrow, deliberate exception, platform-level identity (IAM), not network adjacency, replicate the whole stack per region and let the database — not the backend — carry the multi-region complexity.**

2. Vocabulary — the patterns by name

Term	What it means	Where it shows up here
Core Domain Service (a.k.a. System of Record)	The one service that owns a bounded set of tables and is the only thing allowed to write them. Everyone else reaches that data through its API.	<code>apps/api</code> — owns all 199 Prisma models, including tenant/auth data.
Shared Database (anti-pattern)	Two independently-deployed services both connect straight to the same tables. Named as an anti-pattern because it re-couples services through table shape instead of an explicit, versioned contract.	<code>apps/interview-service</code> connecting to CockroachDB directly via <code>asyncpg</code> — the one deviation found in this codebase.
API Gateway pattern	A single public entry point does authentication/authorization/rate-limiting once, then fans requests out internally. Callers outside the perimeter never see the internal services.	<code>apps/api</code> for <code>guide-agent</code> and <code>manuscript-reviewer</code> today; the target state for <code>interview-service</code> too.
Workload-based decomposition	Split services by <i>resource/latency profile</i> , not by team or by “microservices for their own sake.” A 30s LLM loop, a batch PDF job, and a sub-100ms CRUD call have incompatible autoscaling needs, so they shouldn’t share a deployment.	Why <code>guide-agent</code> (long tool-calling loop), <code>manuscript-reviewer</code> (batch job), and <code>apps/api</code> (CRUD) are three separate Cloud Run resources.
Async Request-Reply (callback) pattern	Caller dispatches work and returns immediately; the worker reports the result back later via its own call, instead of holding the HTTP connection open. Needed because some infra (Cloud Tasks) discards response bodies anyway.	<code>apps/api</code> → <code>guide-agent/manuscript-reviewer</code> : dispatch, then a POST/PATCH callback into <code>apps/api/src/routes/internal/*</code> when done.
Generated client SDK	A thin, typed wrapper around an API contract (OpenAPI/Swagger, or gRPC/protobuf), generated per language, so every caller gets the same behavior “for free” without hand-rewriting logic.	The <i>right</i> way to give a Python service “the same auth/DB logic” as the TypeScript core — generate a Python client from <code>apps/api</code> ’s existing <code>/api-docs</code> OpenAPI spec, don’t reimplement the rules in Python.

What this diagram is saying, plainly:

- **apps/api** is already a correct **Core Domain Service** for two of the three satellites. **guide-agent** and **manuscript-reviewer** hold **zero database credentials** — they get a context snapshot from **apps/api**, do their compute, and report back via a callback. This is the right shape and needed no changes.
- **apps/interview-service** is the deviation: it holds its own **asyncpg** connection and queries CockroachDB directly with hand-written tenant-scoping SQL. This is a live **Shared Database** situation — a second, independently-deployed writer against tables **apps/api** also owns, with its own (separately maintained) copy of the multi-tenant scoping logic.
- Two different internal-auth header names exist (**x-internal-api-key** vs **x-internal-secret**) for what should be one mechanism — an inconsistency, not a deliberate split.
- **No VPC, no Cloud Run IAM invoker configuration exists anywhere** in the deploy pipeline (**infra/cloudbuild.yaml**, **infra/cloudbuild-manuscript-reviewer.yaml**). Every service is a public HTTPS endpoint; **interview-service** is even deployed **--allow-unauthenticated**. The only thing standing between the public internet and these “internal” routes today is a static secret string compared in application code.
- The dashboard’s Websocket connection to **interview-service** bypasses **apps/api** entirely — the one case where the browser talks straight to a satellite service, for real-time audio latency reasons.

4. Target (“ideal”) architecture

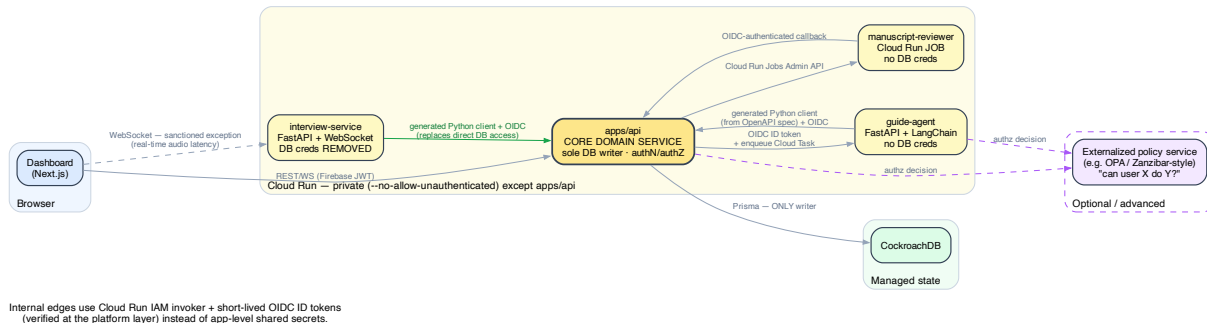


Figure 2: Target architecture

What changes relative to §3:

1. **interview-service gives up its direct DB connection.** It calls **apps/api** (via a generated client, see §5) for whatever tenant/credit data it needs, the same way **guide-agent** already does. This closes the Shared-Database gap and collapses “two copies of the tenant-scoping rule” back down to one.
2. **Every internal call moves from a static shared secret to Cloud Run IAM + OIDC tokens.** Each satellite service is deployed private (**--no-allow-unauthenticated**); only the specific caller identities that need to reach it are granted **roles/run.invoker**, scoped per service — not one blanket secret usable against every internal route at once.
3. **The browser’s Websocket exception stays — deliberately, not by accident.** It’s the

one workload (live audio) where an extra proxy hop through `apps/api` would cost real, user-perceptible latency. Every other frontend path goes through `apps/api`.

4. **(Optional, advanced)** An externalized policy service is shown dashed because it's not a requirement — only worth adding once “does this rule stay consistent across N languages” becomes a real, recurring pain point, not before.

5. “Share the logic” vs. “share the client” — the nuance that matters most

This was the trap in the original question: “*let’s build the auth/DB logic in both JS and Python.*” That’s the wrong thing to duplicate. Split it into two questions:

What	Should it be shared/duplicated across languages?	How
Business & authorization logic — what counts as a valid tenant scope, which role can do what, how a query gets filtered	No — lives in exactly one place, inside the Core Domain Service, in its native language.	<code>apps/api</code> ’s existing <code>BaseRepository</code> /service-layer pattern.
The client that calls that logic over the network	Yes — generate one per language, don’t hand-write N implementations.	Generate from <code>apps/api</code> ’s existing OpenAPI spec (<code>/api-docs</code>) — e.g. <code>openapi-python-client</code> for Python, or move to gRPC/protobuf if you also want streaming + stronger typing across the board.
Authorization <i>decisions</i>, if truly needed locally in multiple languages	Centralize the <i>decision</i> , not the rule text.	An externalized policy service (OPA / Zanzibar-style) — every language asks a tiny “can X do Y?” question instead of encoding the rule.

Why this matters: two hand-written copies of “filter by `organisation_id/tenant_id`” in two languages **will drift** — one gets patched after an incident, the other doesn’t, and now there’s a tenant-isolation bug that only exists in the language nobody was looking at. A generated client can’t drift from its source spec by construction; at worst it’s stale until regenerated, which is a build-time problem, not a silent runtime security bug.

Does this give independent development? **Yes, on exactly one axis: deployment.** Satellite services can ship on their own schedule without redeploying `apps/api`, as long as the API contract is versioned and stable. It does **not** give independent *data ownership* — they stay coupled to the core’s schema and rules, just through an explicit, generated contract instead of a shared table. That’s the correct tradeoff for a platform where nearly everything needs the same tenant-scoping rule (an edtech SaaS), versus a domain that’s genuinely separable.

6. Security angle — what “private Cloud Run” actually buys you

What it protects against:

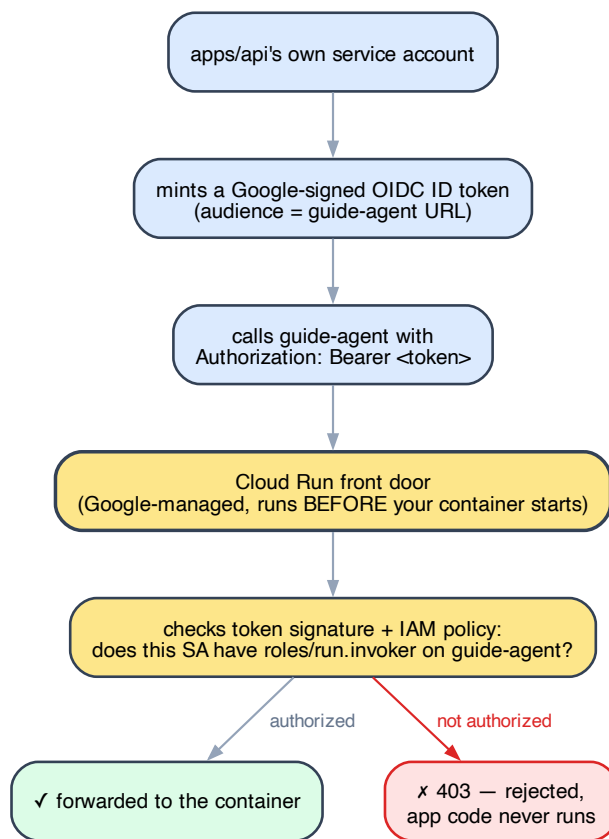


Figure 3: How a private Cloud Run service gets called

- **Anonymous/public access** — a private service (`--no-allow-unauthenticated`) rejects any caller without a valid, IAM-authorized identity, at the platform layer, before your container even boots. Compare: today’s static-secret model relies on every route remembering to check the header correctly — one missed check is an open door.
- **Secret leakage** — there is no long-lived string to leak in logs, source control, or a compromised container’s environment. OIDC tokens are short-lived and minted per-call by the calling service’s own identity.
- **Manual rotation burden** — nothing to rotate; the trust is the IAM binding, not a value.
- **Coarse blast radius** — today, one shared `INTERNAL_API_KEY` value is valid for *every* internal route across every service that checks it. With IAM invoker, each service grants `roles/run.invoker` individually — compromise of one caller’s identity doesn’t automatically grant access to a different service’s internal routes.
- Bonus: every call is now in **Cloud Audit Logs** — “which identity called which private service, when” is a query, not a grep through app logs.

What it does *not* protect against — the things a stakeholder should still ask about even after this migration:

- **SSRF through a still-public endpoint.** If `apps/api` (which stays public, by necessity) has any route that takes a user-supplied URL and fetches it server-side, an attacker can potentially make `apps/api`’s own trusted identity issue requests to private internal services on their behalf. IAM invoker authenticates *who* is calling, not *what* they’re asking a legitimately-authenticated caller to do.
- **Over-broad IAM bindings.** Granting `roles/run.invoker` too widely (e.g. to a whole project instead of one named service account) recreates the “one key unlocks everything” problem in IAM form. Least privilege has to be applied deliberately per binding.
- **What happens *inside* an authorized call.** IAM invoker proves the caller is legitimate; it says nothing about whether that caller’s *payload* is safe (this is exactly why `guide-agent` still needs its own inbound/outbound LLM safety review layer independent of IAM — a different problem, at a different layer).
- **Cross-region requests** aren’t network-isolated to a VPC just because they’re IAM-authenticated — see §7 on why that’s fine for the auth question but not for latency.

Defense in depth, stacked correctly:

1. IAM invoker + OIDC (identity — *who* is calling) — the primary fix here.
2. Per-service least-privilege bindings (blast radius containment).
3. (*Optional hardening layer, not the primary mechanism*) Serverless VPC Access connector / Private Google Access, if you also want to restrict network egress paths, not just identity.
4. Payload-level checks specific to the workload (e.g. `guide-agent`’s LLM reviewers, request validation, rate/budget limits) — orthogonal to all of the above.

7. Multi-region — why the database is the hard part, not the backend services

Two things that were conflated in the original question, worth separating cleanly:

Same-region deployment (already true here — everything sits in `asia-south1`) is about latency. Two Cloud Run services in the same region are physically closer, so round-trips are faster. This is real and worth keeping for any *synchronous* call chain (a user-facing request that blocks on a satellite service’s answer).

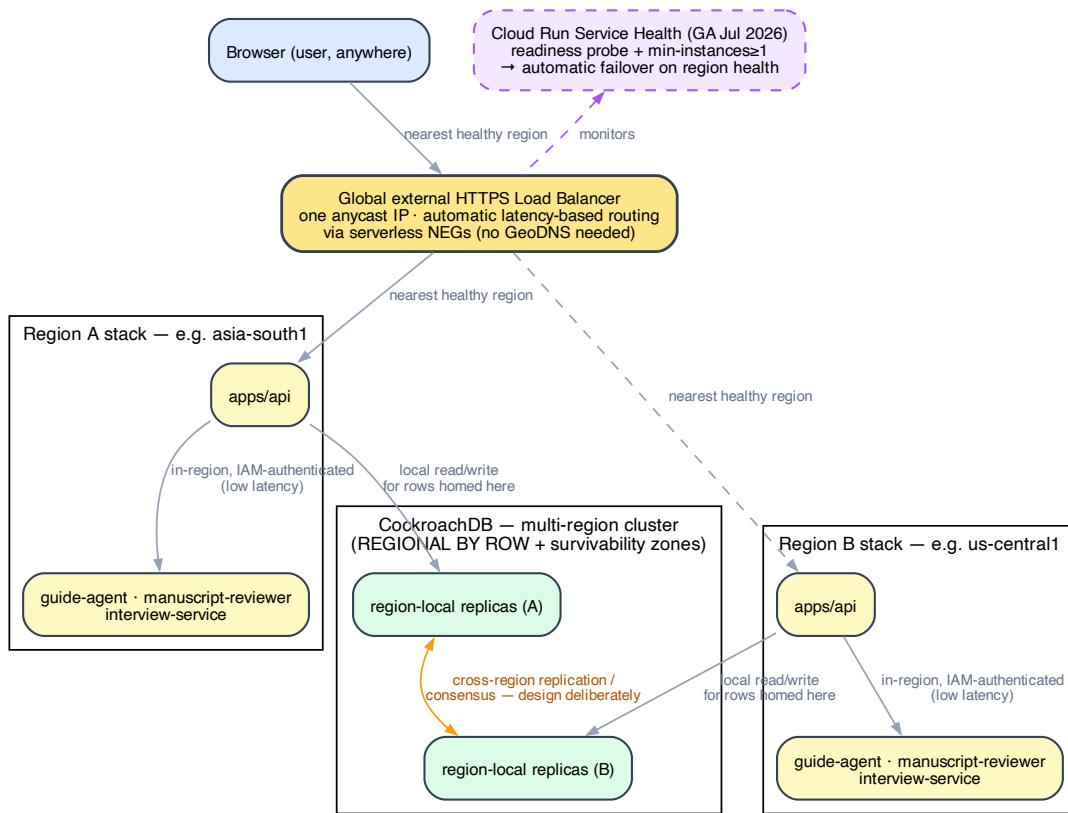


Figure 4: Multi-region deployment

Same-VPC is *not* what makes cross-service calls fast or secure. Cloud Run-to-Cloud-Run traffic already rides Google’s internal backbone even when addressed by public HTTPS URLs — a VPC connector doesn’t meaningfully change that latency. Its actual value is a *second, independent* trust boundary (network-level, on top of the identity-level one from §6) — worth adding once IAM invoker is in place, not instead of it.

If this platform ever needs true multi-region (e.g. serving users on two continents):

- **Don’t split one call chain across regions.** Deploying `apps/api` in region A and `guide-agent` in region B just because “they’re separate services” turns every synchronous internal call into a slow, chatty cross-WAN round trip for no benefit. Replicate the *whole* tightly-coupled stack (core + its satellites) per region instead — that’s what the diagram shows.
- **The `async-dispatch` workloads tolerate cross-region latency fine.** `guide-agent`’s turn dispatch and `manuscript-reviewer`’s job trigger are already `async-with-callback` (§2) — nothing is blocking a live user request during that round trip, so these two could in principle run centrally even if the rest of the stack is regional, if there’s a reason to (e.g. GPU/model availability only in one region). That’s a legitimate exception to “replicate the whole stack,” made deliberately — the same category of exception as the `WebSocket-direct` case in §4.
- **The real bottleneck is CockroachDB, not the backend.** Standing up a second `apps/api` in a new region does nothing for users if every query still has to round-trip to a single-region database. CockroachDB is *designed* for exactly this problem — multi-region survivability zones and **REGIONAL BY ROW** tables, which pin each row to a home region so same-region reads/writes stay local. Going multi-region without configuring this first just relocates the latency bottleneck from “which backend region” to “every DB call,” which is worse, not better.
- **IAM invoker auth (§6) is region-agnostic** — it doesn’t need same-VPC or same-region to work, so it’s not a blocker to any of this; it’s actually a prerequisite, since it’s the mechanism that lets two regional stacks trust each other’s calls without needing to be network-adjacent.
- **Routing:** a global HTTPS load balancer in front, sending each user to their nearest full regional stack — see §7.1, this part turns out to require less manual work than it sounds like.

7.1 The Global Load Balancer’s region-routing is automatic — verified against current GCP docs (2026-09)

This was a specific claim worth checking rather than assuming, since it changes how much of §7 is “design work” vs. “just deploy it”: **yes, a Google Cloud external HTTPS Load Balancer routes each request to the nearest healthy regional backend automatically, with no GeoDNS or manual region-selection logic required.**

How it actually works:

- The load balancer is fronted by **one global anycast IP** — the same IP is announced from every Google edge location worldwide, so a user’s request simply enters Google’s network at whichever edge is topologically closest to them. This is a step ahead of DNS-based geo-routing (e.g. Route 53 latency records): there’s only one DNS record regardless of region count, and failover doesn’t wait on DNS TTL/caching.
- Once inside Google’s network, if the **backend service has multiple serverless NEG’s attached — one per region** — the load balancer forwards each request to the NEG in the closest available region. A user in Sydney hits `australia-southeast1`; a user in Frankfurt hits `europa-west1`; a user in Chicago hits `us-central1` — automatically, per-request, with no application code or DNS config making that decision.
- **As of July 2026, this got a real upgrade: Cloud Run Service Health went GA**, adding

automatic cross-region failover on top of the existing latency routing. Setup is two steps — add a readiness probe, set `min-instances 1` — after which the load balancer reads each NEG’s health and reroutes away from a region automatically once enough instances start failing their readiness probes, without anyone flipping a switch by hand.

So the “automatic” part of your question is genuinely true for the routing decision — you don’t hand-write proximity logic, and you get health-aware failover essentially for free once the readiness probe is wired up. What’s still real, deliberate work, not automated by any of this:

1. **Actually deploying the service to each target region** in the first place (a regional Cloud Run deployment per region, as already covered in §7’s stack-replication point) and adding each as a serverless NEG to the *same* backend service — a one-time infra setup task (static IP, managed cert, NEGs, URL map), not zero-effort.
2. **The database is still the bottleneck this doesn’t touch.** The load balancer solves “*which region should this browser’s request land in*” — it does nothing for “*once it lands, how fast can that region’s `apps/api` reach the data it needs.*” Route a Bangalore user perfectly to the nearest healthy `apps/api` region, and if that region still has to cross-region round-trip to a single-region CockroachDB for every query, the bottleneck has only moved, not disappeared — this is exactly why §7’s REGIONAL BY ROW point remains the harder half of going multi-region, not a secondary concern.
3. **This automatic routing is a *public-entry-point* pattern.** It applies to whichever service sits behind the public global LB (typically `apps/api` and/or the frontend). It doesn’t automatically extend to internal, private backend-to-backend calls between satellite services (§6) — a regional `apps/api` calling `guide-agent` should still call the co-located regional instance directly via IAM-authenticated Cloud Run invocation, not be routed through a public global LB, since those services are meant to stay private. (GCP does have a separate cross-region *internal* Application Load Balancer product for private multi-region service-to-service traffic, if that specific need ever comes up — noted here as a pointer, not evaluated in depth.)

Sources: [Set up a global external Application Load Balancer with Cloud Run \(GCP docs\)](#), [Cloud Run NEGs and Global HA](#), [How to Set Up Cloud Run Multi-Region Deployment with Global Load Balancing](#)

8. Recap — what was verified vs. assumed, across the whole discussion

Claim examined	Verdict
“Manuscript-reviewer connects to CockroachDB”	Wrong — zero DB access confirmed by reading the code; only <code>interview-service</code> does. (My own diagram was ambiguous and caused the confusion.)
“All services run in the same region”	True — <code>_DEPLOY_REGION: asia-south1</code> in both <code>cloudbuild.yaml</code> and <code>cloudbuild-manuscript-reviewer.yaml</code> .
“All services run in the same VPC”	False — no VPC connector, no private ingress config anywhere; every service is public HTTPS with app-level shared-secret auth.

Claim examined	Verdict
“One API-layer service should own DB ops; others call it”	Right instinct — this is the standard Core Domain Service pattern, and it’s already true for <code>guide-agent/manuscript-reviewer</code> ; <code>interview-service</code> is the one gap.
“Build shared auth/DB logic in both JS and Python”	Right goal, wrong mechanism — share a <i>generated client</i> from one contract, don’t hand-duplicate the <i>rules</i> per language (§5). Also a hard blocker as literally stated: <code>packages/database/packages/helpers</code> are TypeScript-only and cannot be <code>import</code> -ed from Python regardless of design.
“Frontend always calls <code>apps/api</code> , which calls the needed backend service”	Right, with one sanctioned exception — the WebSocket path to <code>interview-service</code> , kept deliberately for real-time audio latency.
“Backend services don’t need to share a VPC with <code>apps/api</code> if same region”	Right conclusion, for the wrong reason as originally framed — the actual enabler isn’t “same region is good enough,” it’s that trust should be IAM/OIDC-based , which works regardless of network adjacency; same-region remains valuable purely for latency, independent of the auth question.
“Keep internal Cloud Run services private, let <code>apps/api</code> call them”	Right — this is exactly Cloud Run IAM invoker . (Minor correction: the deploy flag is <code>--no-allow-unauthenticated</code> , not <code>--allow-no-authenticated</code> .)
“Deploying to another geography, deploy everything there too”	Right instinct — replicate the whole tightly-coupled stack per region; the part that actually needs deliberate design is the database’s multi-region configuration, not the backend services.
“A global load balancer would automatically send traffic to the nearest regional deployment, so multi-region traffic handling is easy”	Verified true for the routing decision itself (§7.1) — one anycast IP, automatic latency-based routing via serverless NEG’s, plus GA-since-July-2026 health-aware auto-failover. Not true that it makes multi-region “easy” overall — the LB removes the <i>routing</i> problem, not the <i>database</i> problem, which remains the harder half.

9. If this were ever acted on — rough order of operations

Not a commitment, just what the dependency order would look like if the ideal state in §4 were pursued:

1. Fix the internal-auth header inconsistency (`x-internal-api-key` vs `x-internal-secret`) — cheap, no architecture change.

2. Move all internal service-to-service calls from static secrets to Cloud Run IAM invoker + OIDC — unlocks real security gains immediately, independent of anything else here.
3. Generate a Python client from `apps/api`'s OpenAPI spec; migrate `guide-agent`'s existing internal calls to use it (low risk — it already has no DB access, this is a refactor of *how* it calls, not *whether* it's allowed to).
4. Use that same generated client to remove `interview-service`'s direct `asyncpg`/CockroachDB connection, replacing its tenant-scoping SQL with calls back into `apps/api`.
5. Only then, if cross-language authorization drift becomes a recurring real problem (not preemptively): evaluate an externalized policy service.
6. Multi-region is a much later, separate initiative — gated on a real geographic-latency requirement showing up. The load-balancer/routing half (§7.1) is comparatively cheap and mostly-automatic once you decide to do it; start the real design effort on the CockroachDB REGIONAL BY ROW side, not the backend topology.

10. Cloud Run building block: Service vs. Job vs. Function

Every deployable discussed in this note is one of these three shapes. Worth being precise about the difference, because as of **August 2024 they stopped being separate products** — Google renamed Cloud Functions to **Cloud Run functions** and folded it into the Cloud Run platform; a “function” today is, under the hood, a Cloud Run service where Google builds the container for you instead of you writing a Dockerfile. So the real question was never “which GCP product” — it's **which lifecycle shape fits the workload**, on infrastructure that's now unified.

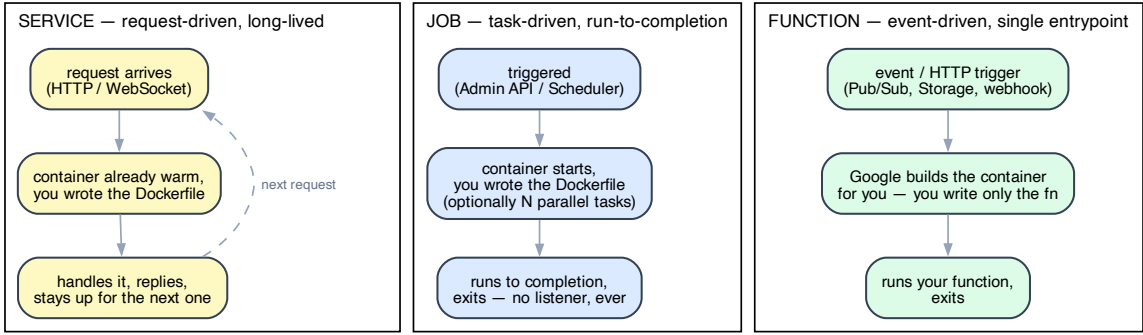


Figure 5: The three Cloud Run lifecycle shapes

	Service	Job	Function
Lifecycle	Long-lived, request-driven — stays warm, handles many requests over time	Ephemeral, task-driven — starts, runs to completion, exits. No listener, no port, ever.	Ephemeral, event-driven — one trigger in, one invocation out

	Service	Job	Function
Who builds the container	You do (your own Dockerfile — full control of run-time/deps/language)	You do	Google does — you supply only the function source
Trigger	HTTP request / WebSocket connection	Manual, Scheduler, or another service calling the Cloud Run Jobs Admin API (runJob) — never an HTTP call to the job itself	HTTP, Pub/Sub, Cloud Storage events, and other event sources
Max duration	60 minutes per request (default 5 min)	Up to 168 hours (7 days) per task (default 10 min; GPU tasks cap at 1 hour)	60 minutes (2nd gen); legacy 1st-gen functions cap at 9 min
Parallelism	Autoscales instances to handle concurrent requests	Can fan out N parallel task indices in one execution — a native batch-array primitive	One invocation per event; scales by concurrent events
Reachable by the frontend?	Yes, if public	Never directly — no HTTP surface exists to reach	Yes, if HTTP-triggered and public

The decision, as three questions, in order:

1. **Does it need to listen and respond to requests indefinitely?** → **Service**. (apps/api, guide-agent, interview-service — all long-lived listeners.)
2. **No listener at all — does it run a task to completion, possibly for hours, possibly with parallel task fan-out?** → **Job**. (manuscript-reviewer — reads one PDF from GCS, reviews it, PATCHes a result back, exits. No HTTP surface was ever needed, and a single manuscript review could plausibly run well past a function’s 60-minute ceiling — Jobs’ 7-day headroom is exactly why this wasn’t built as a function.)
3. **Small, single-purpose, triggered by an event, and you don’t want to own a Dockerfile for it?** → **Function**. (The cloud-functions/ workspace — audio-explainer, latex-converter, prompt-eval, solution-agent, tts-generator — five narrow, single-entypoint transforms, each easier to express as “here’s my function” than as a maintained container image.)

A useful reframe if the three ever feel interchangeable: a **Function** is a Service you didn’t have to containerize, scoped to one entypoint and a shorter timeout; a **Job** is the one shape of the three with **no HTTP surface at all** — if something has no business being reachable by a request, Job is the only one of the three that structurally enforces that, rather than relying on IAM/network config to keep it private (§6).

10.1 “It’s async, so it should be a Function” — a common mix-up, worth untangling

Async dispatch and compute shape are two independent decisions, not one. Cloud Tasks (the queue that gives you reliable, retried delivery) will call *any* HTTPS target — a Service route, a

Function, doesn't matter. Picking "async" doesn't pick the compute shape for you; the §10 questions (duration, does it need a listener, does it need independent scaling) still decide that, same as for a synchronous call.

Proof from this codebase: `apps/api` dispatches to `guide-agent` via exactly this pattern — enqueue a Cloud Task, it POSTs `guide-agent`'s `/turn` endpoint, the answer comes back later via a callback. It's fully async — and `guide-agent` is a **Service**, not a Function, because the work (a multi-minute, multi-tool-call LLM loop) needs more time and more persistent state than a Function's model comfortably gives.

Also worth being precise on: **Cloud Tasks stores the *queue of pending invocations* (retries, backoff, dedup) — not the *result*.** Whoever processes the task still has to persist the outcome itself (here: the callback route writing back into `apps/api`'s DB). Don't reach for "Cloud Tasks will store it" as a substitute for actually writing the result somewhere.

Where "async → Function" *is* a good instinct: when the async task is genuinely small, stateless, single-purpose (resize an image, fire a webhook) — Function fits because it's **small**, not because it's async. And when the async work is one-shot batch with no need to ever be reached by a request at all (`manuscript-reviewer`), it skips Cloud Tasks entirely and dispatches via the **Jobs Admin API** instead — a third mechanism, used because Jobs have no HTTP surface for a task queue to call in the first place.

10.2 “It's a small/quick task, so it should be a Function” — necessary, not sufficient

Fitting under a Function's 60-minute cap only rules Job *out*; it doesn't automatically pick Function over “just a route on an existing Service.” Two further questions decide it:

Per-item or batch? This platform's own `ai-evaluator` Cloud Function is the confirming example: `exam/submission` evaluation is dispatched **one Cloud Task per submission** onto an `exam-evaluation-queue`, each landing as one independent, stateless Function invocation — exactly the right shape for “evaluate this one thing.” But if the need ever becomes “*evaluate a whole batch together as one operation*” (e.g. grade all 500 submissions in an exam window in one run), that's a better fit for a **Job**'s native parallel task fan-out — one execution ID, one completion signal, built-in concurrency control — rather than firing N independent Cloud-Tasks-to-Function calls and having to build batch-level bookkeeping (did all 500 finish? which failed?) yourself on top.

Does it need its own deployable at all? Being quick doesn't require a *new* Function — it could just be a route on an existing Service (§10.1's callback-route pattern). The real reason `ai-evaluator/prompt-eval` are their own Functions rather than routes on `apps/api` is almost certainly that they carry their own LLM-call dependency footprint and want a timeout/failure domain independent of the main API's request budget — that's the actual test, not “is it small.”

Sources: [Cloud Run Jobs vs. Cloud Functions — key differences](#), [The Unification of Google Cloud Functions and Google Cloud Run](#), [Compare Cloud Run functions \(GCP docs\)](#), [Cloud Run Quotas and Limits \(GCP docs\)](#)